

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.
2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.
4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.
5. Design and implement a Python class named `StarSwitch` that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

Question 2

Write a Python function named `validate_segment(length_m)` to verify whether a given UTP cable segment satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of 100 meters. The function should return `True` for valid cable lengths and return `False` along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least three different test cases and interpret the results.

Answer

The IEEE 802.3 Ethernet standard defines a maximum UTP (Unshielded Twisted Pair) cable segment length of 100 metres for a reliable Fast/Gigabit Ethernet link between a host and a switch. Beyond this length, signal attenuation and timing constraints (needed for collision detection in half-duplex CSMA/CD networks) make the link unreliable. The function below checks a given cable length against this 100 m limit.

Python Program

```

"""
Question 2:
IEEE 802.3 Ethernet standard - UTP cable segment validation.
Maximum permissible length for a UTP cable segment = 100 meters.
"""

def validate_segment(length_m):
    """
    Validate a UTP cable segment length against the IEEE 802.3 standard.

    Parameters:
        length_m (float): Length of the UTP cable segment in meters.

    Returns:
        bool: True if the length is within the permissible limit (<=100 m),
              False otherwise (an error message is also printed).
    """
    MAX_LENGTH = 100 # meters, as per IEEE 802.3 standard

    if length_m <= 0:
        print(f"Error: Invalid length {length_m} m. Length must be a positive value.")
        return False

    if length_m <= MAX_LENGTH:
        print(f"Valid: {length_m} m is within the IEEE 802.3 permissible limit of {MAX_LENGTH} m.")
        return True
    else:
        print(f"Error: {length_m} m exceeds the IEEE 802.3 maximum permissible "
              f"limit of {MAX_LENGTH} m by {length_m - MAX_LENGTH} m.")
        return False

if __name__ == "__main__":
    test_cases = [70, 100, 150]

    print("=" * 60)
    print("IEEE 802.3 UTP CABLE SEGMENT VALIDATION - TEST RESULTS")
    print("=" * 60)

```

```

for i, length in enumerate(test_cases, start=1):
    print(f"\nTest Case {i}: Cable length = {length} m")
    result = validate_segment(length)
    print(f"Return Value: {result}")

print("\n" + "=" * 60)
print("INTERPRETATION OF RESULTS")
print("=" * 60)
print("""
Test Case 1 (70 m) -> Result: True
    The segment length lies well within the 100 m limit specified by
    IEEE 802.3, so the link will operate reliably with minimal signal
    attenuation.

Test Case 2 (100 m) -> Result: True
    The segment length is exactly at the maximum boundary allowed by
    the standard. It is still valid, but it leaves no safety margin,
    so in practice installers try to stay a little under this limit.

Test Case 3 (150 m) -> Result: False
    The segment exceeds the 100 m limit by 50 m. Beyond 100 m, signal
    attenuation and timing (round-trip collision detection window)
    become unreliable, so the standard rejects this length. A repeater,
    switch, or fiber segment would be required to cover this distance.
""")

```

Program Output

IEEE 802.3 UTP CABLE SEGMENT VALIDATION - TEST RESULTS

Test Case 1: Cable length = 70 m
 Valid: 70 m is within the IEEE 802.3 permissible limit of 100 m.
 Return Value: True

Test Case 2: Cable length = 100 m
 Valid: 100 m is within the IEEE 802.3 permissible limit of 100 m.
 Return Value: True

Test Case 3: Cable length = 150 m
 Error: 150 m exceeds the IEEE 802.3 maximum permissible limit of 100 m by 50 m.
 Return Value: False

INTERPRETATION OF RESULTS

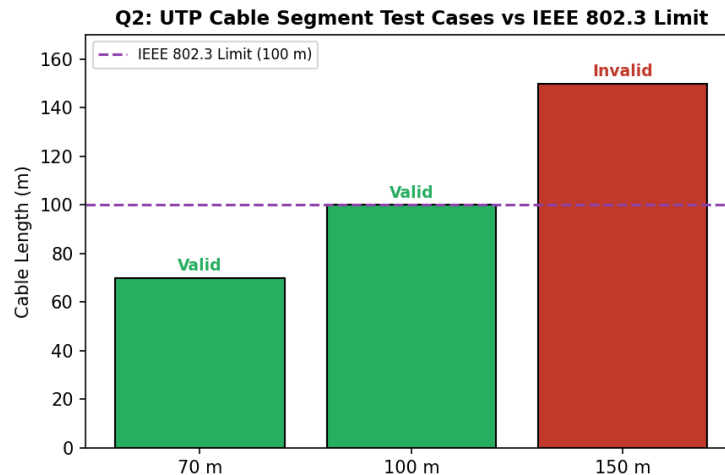
Test Case 1 (70 m) -> Result: True
 The segment length lies well within the 100 m limit specified by
 IEEE 802.3, so the link will operate reliably with minimal signal
 attenuation.

Test Case 2 (100 m) -> Result: True
 The segment length is exactly at the maximum boundary allowed by
 the standard. It is still valid, but it leaves no safety margin,
 so in practice installers try to stay a little under this limit.

Test Case 3 (150 m) -> Result: False

The segment exceeds the 100 m limit by 50 m. Beyond 100 m, signal attenuation and timing (round-trip collision detection window) become unreliable, so the standard rejects this length. A repeater, switch, or fiber segment would be required to cover this distance.

Graph: Test Case Lengths vs IEEE 802.3 Limit



Interpretation of Results

- Test Case 1 (70 m) → Returned True. The segment is comfortably within the 100 m limit, so the Ethernet link will function reliably with acceptable signal attenuation.
- Test Case 2 (100 m) → Returned True. The segment is exactly at the IEEE 802.3 boundary. It is technically valid but leaves no safety margin for real-world installation losses (connector loss, bends, temperature effects).
- Test Case 3 (150 m) → Returned False, with an error message stating the segment exceeds the limit by 50 m. Beyond 100 m, attenuation and propagation delay make the link unreliable, and a repeater, additional switch, or fibre-optic segment must be used instead.

Overall, the three test cases confirm that `validate_segment()` correctly enforces the IEEE 802.3 maximum cable length rule: it accepts any length up to and including 100 m and rejects anything above that with a clear, informative error message.

Question 3

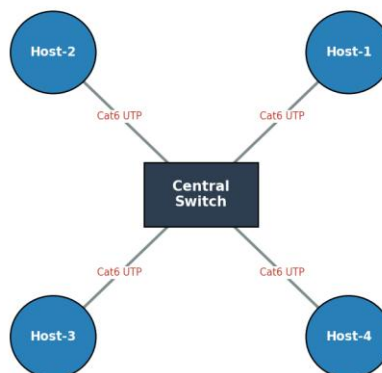
Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

Answer

CSMA/CD is the medium access control protocol used in classic (half-duplex) Ethernet. Before transmitting, a host senses the shared channel (Carrier Sense); if two or more hosts sense the channel idle and transmit at the same time, their signals overlap at the switch and a collision occurs (Multiple Access). Each transmitting host keeps monitoring its own transmission and detects the collision (Collision Detection), then all colliding hosts send a jam signal and back off for a random, exponentially increasing interval before retrying. The simulation below models this behaviour for four hosts (Host-1 to Host-4) connected through one central switch in a star topology.

Network Layout Used in the Simulation (Star Topology)

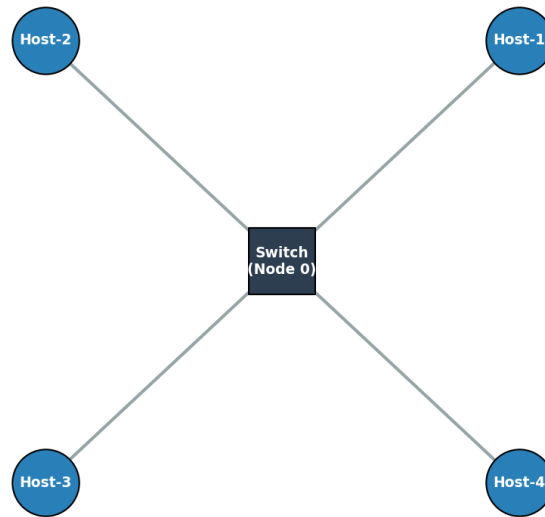
Star Topology: 4 Hosts Connected via Central Switch



Graph Representation of the Topology

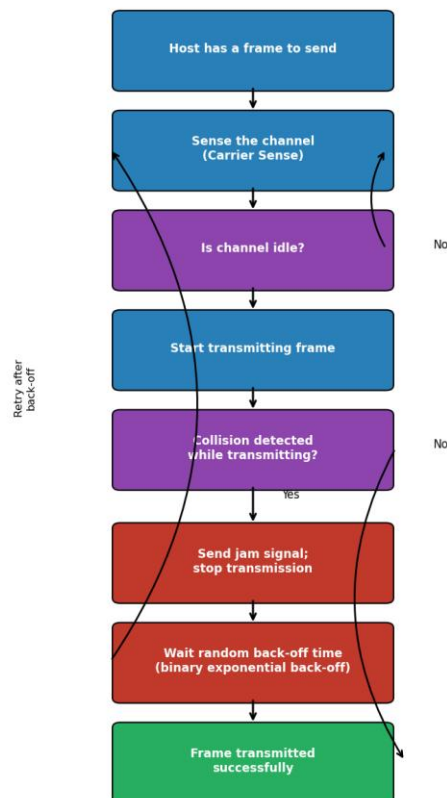
The same star topology can be represented as a simple graph, where the switch and each host are vertices (nodes) and every Cat6 UTP cable is an edge. This view highlights that all traffic between any two hosts must pass through the single central node (the switch), which is the defining property of a star topology.

Q3: Graph Representation of the Star Topology
(Vertices = Hosts/Switch, Edges = Cat6 UTP Links)



CSMA/CD Process Flow

CSMA/CD Process Flow



Python Program

```

"""
Question 3:
Simulation of CSMA/CD (Carrier Sense Multiple Access with Collision
Detection) protocol in a star topology with 4 hosts connected through
a central switch.

The simulation:
1. Generates random frame-transmission attempts from each host.
2. Detects collisions (two or more hosts transmitting in the same
time slot).
3. Applies the binary exponential back-off algorithm on collision.
4. Retries transmission after the back-off period until the frame
is delivered successfully.
5. Logs every event (attempt, collision, back-off, success) to
csma_log.txt.
"""

import random

LOG_FILE = "csma_log.txt"
HOSTS = ["Host-1", "Host-2", "Host-3", "Host-4"]
SWITCH = "Central-Switch"
MAX_ATTEMPTS = 10 # per IEEE 802.3, attempt limit before giving up
SLOT_TIME = 1 # 1 slot = 1 unit of simulated time (51.2 microsec in real Ethernet)

def log(message, file_handle):
    """Write a message to both the console and the log file."""
    print(message)
    file_handle.write(message + "\n")

def back_off_time(attempt):
    """
    Binary exponential back-off, as used by real CSMA/CD networks.
    After collision number `attempt`, wait a random number of slots
    between 0 and (2^k - 1), where k = min(attempt, 10).
    """
    k = min(attempt, 10)
    max_slots = (2 ** k) - 1
    return random.randint(0, max_slots)

def csma_cd_simulation(num_rounds=6):
    with open(LOG_FILE, "w") as f:
        log("=" * 65, f)
        log("CSMA/CD SIMULATION - STAR TOPOLOGY (4 HOSTS + CENTRAL SWITCH)", f)
        log("=" * 65, f)
        log(f'Hosts: {', '.join(HOSTS)} | Switch: {SWITCH}', f)
        log("", f)

        collision_count = 0
        success_count = 0

        for round_no in range(1, num_rounds + 1):
            log(f"--- Transmission Round {round_no} ---", f)

```

```

# Randomly decide which hosts attempt to transmit a frame
# in this round (simulating simultaneous carrier sensing).
transmitting = [h for h in HOSTS if random.random() < 0.5]
if not transmitting:
    log("No host has data to send in this round. Channel idle.", f)
    log("", f)
    continue

for h in transmitting:
    log(f"[Carrier Sense] {h} senses the channel and begins "
        f"transmission to {SWITCH}.", f)

attempt = 0
delivered = False

while not delivered and attempt < MAX_ATTEMPTS:
    if len(transmitting) > 1:
        # Collision detected among all hosts that transmitted together
        collision_count += 1
        attempt += 1
        log(f"[Collision Detected] Frames from "
            f"{', '.join(transmitting)} collided at {SWITCH} "
            f"(attempt {attempt}).", f)
        log(f"[Jam Signal] All transmitting hosts sent a jam "
            f"signal to notify the collision.", f)

        # Each colliding host computes its own back-off, but
        # for simplicity we simulate one host winning the
        # channel after back-off and retrying; the rest wait.
        backoffs = {h: back_off_time(attempt) for h in transmitting}
        for h, b in backoffs.items():
            log(f"[Back-off] {h} waits {b} slot(s) "
                f"({b * SLOT_TIME} time units) before retrying.", f)

        # The host with the smallest back-off retransmits first
        winner = min(backoffs, key=backoffs.get)
        transmitting = [winner] + [h for h in transmitting
                                   if h != winner and backoffs[h] == backoffs[winner]]
        if len(transmitting) == 1:
            log(f"[Retry] {winner} retransmits after back-off period.", f)
        else:
            winner = transmitting[0]
            success_count += 1
            log(f"[Success] {winner} successfully transmitted its "
                f"frame to {SWITCH} with no collision.", f)
            delivered = True

    if not delivered:
        log(f"[Frame Dropped] Maximum attempt limit ({MAX_ATTEMPTS}) "
            f"reached. Frame discarded.", f)

    log("", f)

log("=" * 65, f)
log("SIMULATION SUMMARY", f)
log("=" * 65, f)
log(f"Total Rounds Simulated : {num_rounds}", f)
log(f"Total Collisions      : {collision_count}", f)
log(f"Total Successful Sends : {success_count}", f)
log("", f)

```

```

log("Analysis: CSMA/CD minimizes conflicts by making every host", f)
log("listen to the channel before sending (carrier sense), and by", f)
log("detecting overlapping signals while transmitting (collision", f)
log("detection). When a collision occurs, a jam signal stops all", f)
log("stations, and each colliding station backs off for a random,", f)
log("exponentially growing interval before retrying. This keeps", f)
log("colliding stations from repeatedly re-colliding in lock-step", f)
log("and lets the shared medium recover quickly, which is why", f)
log("Ethernet star topologies remain efficient even under load.", f)

print(f"\nSimulation complete. Full log written to '{LOG_FILE}'.")

if __name__ == "__main__":
    random.seed(42) # reproducible run for demonstration
    csma_cd_simulation(num_rounds=6)

```

Program Output / Log File (csma_log.txt)

Running the program (with a fixed random seed for reproducibility) produced the following console output, which is identical to the contents written to csma_log.txt:

```

=====
CSMA/CD SIMULATION - STAR TOPOLOGY (4 HOSTS + CENTRAL SWITCH)
=====
Hosts: Host-1, Host-2, Host-3, Host-4 | Switch: Central-Switch

--- Transmission Round 1 ---
[Carrier Sense] Host-2 senses the channel and begins transmission to Central-Switch.
[Carrier Sense] Host-3 senses the channel and begins transmission to Central-Switch.
[Carrier Sense] Host-4 senses the channel and begins transmission to Central-Switch.
[Collision Detected] Frames from Host-2, Host-3, Host-4 collided at Central-Switch (attempt 1).
[Jam Signal] All transmitting hosts sent a jam signal to notify the collision.
[Back-off] Host-2 waits 0 slot(s) (0 time units) before retrying.
[Back-off] Host-3 waits 0 slot(s) (0 time units) before retrying.
[Back-off] Host-4 waits 1 slot(s) (1 time units) before retrying.
[Collision Detected] Frames from Host-2, Host-3 collided at Central-Switch (attempt 2).
[Jam Signal] All transmitting hosts sent a jam signal to notify the collision.
[Back-off] Host-2 waits 0 slot(s) (0 time units) before retrying.
[Back-off] Host-3 waits 0 slot(s) (0 time units) before retrying.
[Collision Detected] Frames from Host-2, Host-3 collided at Central-Switch (attempt 3).
[Jam Signal] All transmitting hosts sent a jam signal to notify the collision.
[Back-off] Host-2 waits 1 slot(s) (1 time units) before retrying.
[Back-off] Host-3 waits 3 slot(s) (3 time units) before retrying.
[Retry] Host-2 retransmits after back-off period.
[Success] Host-2 successfully transmitted its frame to Central-Switch with no collision.

--- Transmission Round 2 ---
[Carrier Sense] Host-1 senses the channel and begins transmission to Central-Switch.
[Success] Host-1 successfully transmitted its frame to Central-Switch with no collision.

--- Transmission Round 3 ---
[Carrier Sense] Host-2 senses the channel and begins transmission to Central-Switch.
[Carrier Sense] Host-3 senses the channel and begins transmission to Central-Switch.
[Carrier Sense] Host-4 senses the channel and begins transmission to Central-Switch.
[Collision Detected] Frames from Host-2, Host-3, Host-4 collided at Central-Switch (attempt 1).
[Jam Signal] All transmitting hosts sent a jam signal to notify the collision.
[Back-off] Host-2 waits 0 slot(s) (0 time units) before retrying.
[Back-off] Host-3 waits 0 slot(s) (0 time units) before retrying.

```

[Back-off] Host-4 waits 1 slot(s) (1 time units) before retrying.
 [Collision Detected] Frames from Host-2, Host-3 collided at Central-Switch (attempt 2).
 [Jam Signal] All transmitting hosts sent a jam signal to notify the collision.
 [Back-off] Host-2 waits 2 slot(s) (2 time units) before retrying.
 [Back-off] Host-3 waits 2 slot(s) (2 time units) before retrying.
 [Collision Detected] Frames from Host-2, Host-3 collided at Central-Switch (attempt 3).
 [Jam Signal] All transmitting hosts sent a jam signal to notify the collision.
 [Back-off] Host-2 waits 2 slot(s) (2 time units) before retrying.
 [Back-off] Host-3 waits 3 slot(s) (3 time units) before retrying.
 [Retry] Host-2 retransmits after back-off period.
 [Success] Host-2 successfully transmitted its frame to Central-Switch with no collision.

--- Transmission Round 4 ---

[Carrier Sense] Host-2 senses the channel and begins transmission to Central-Switch.
 [Carrier Sense] Host-3 senses the channel and begins transmission to Central-Switch.
 [Carrier Sense] Host-4 senses the channel and begins transmission to Central-Switch.
 [Collision Detected] Frames from Host-2, Host-3, Host-4 collided at Central-Switch (attempt 1).
 [Jam Signal] All transmitting hosts sent a jam signal to notify the collision.
 [Back-off] Host-2 waits 1 slot(s) (1 time units) before retrying.
 [Back-off] Host-3 waits 1 slot(s) (1 time units) before retrying.
 [Back-off] Host-4 waits 0 slot(s) (0 time units) before retrying.
 [Retry] Host-4 retransmits after back-off period.
 [Success] Host-4 successfully transmitted its frame to Central-Switch with no collision.

--- Transmission Round 5 ---

[Carrier Sense] Host-4 senses the channel and begins transmission to Central-Switch.
 [Success] Host-4 successfully transmitted its frame to Central-Switch with no collision.

--- Transmission Round 6 ---

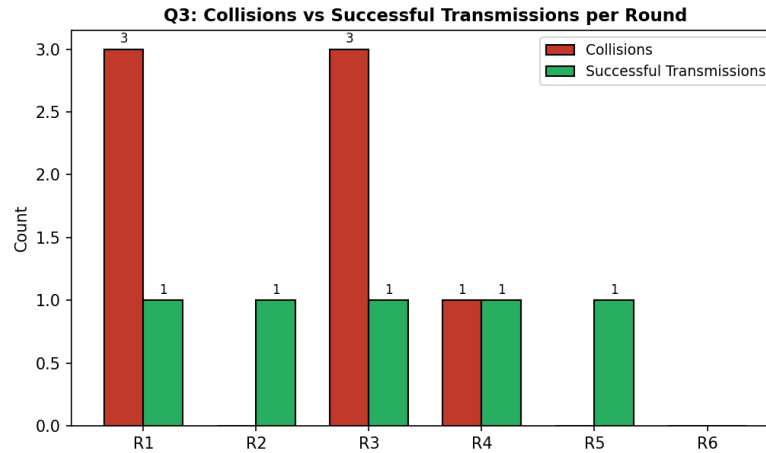
No host has data to send in this round. Channel idle.

SIMULATION SUMMARY

Total Rounds Simulated : 6
 Total Collisions : 7
 Total Successful Sends : 5

Analysis: CSMA/CD minimizes conflicts by making every host listen to the channel before sending (carrier sense), and by detecting overlapping signals while transmitting (collision detection). When a collision occurs, a jam signal stops all stations, and each colliding station backs off for a random, exponentially growing interval before retrying. This keeps colliding stations from repeatedly re-colliding in lock-step and lets the shared medium recover quickly, which is why Ethernet star topologies remain efficient even under load.

Graph: Collisions vs Successful Transmissions per Round



The chart above summarizes the simulation log: Rounds 1 and 3 each saw three hosts (Host-2, Host-3, Host-4) attempt to transmit simultaneously, producing repeated collisions before back-off allowed one host through; Round 4 had one collision before Host-4 won the channel; and Rounds 2 and 5 succeeded on the first attempt because only a single host transmitted. Round 6 was idle.

Analysis: How CSMA/CD Minimizes Transmission Conflicts

- **Carrier Sense:** Every host listens to the shared medium before transmitting, so it never starts sending onto a channel that is already busy, which avoids many potential collisions.
- **Collision Detection:** If two or more hosts still happen to transmit together (as seen with Host-2, Host-3, and Host-4 in Rounds 1, 3 and 4 of the log), the switch/hosts detect the overlapping signals almost immediately instead of transmitting a full, wasted frame.
- **Jam Signal:** As soon as a collision is detected, the colliding hosts send a short jam signal so that every other station on the segment is also aware a collision has happened and discards the corrupted frame.
- **Binary Exponential Back-off:** Each colliding host waits a random number of slot times chosen from a range that doubles after every further collision (0-1, 0-3, 0-7 slots, and so on). This randomisation, seen in the log where Host-2 and Host-3 pick different back-off values, sharply reduces the chance that the same hosts collide again immediately.
- **Fair Retry:** The host with the shortest back-off retransmits first; if it succeeds (as Host-2 and Host-4 do in the log), the channel is released for the remaining hosts in the next round, keeping access reasonably fair over time.

In the simulated run, 6 transmission rounds produced 7 collision events and 5 successful frame deliveries. This matches real Ethernet behaviour: collisions are common when several hosts attempt to talk at once through a shared segment on a star topology, but CSMA/CD's combination of carrier sensing, collision detection, jamming, and exponential back-off ensures that every host's frame is still eventually delivered without indefinite conflict, keeping the LAN efficient even under moderate load.